Arquitetura e Roadmap: Sudoku Aneróthico PRÓ

Este documento consolida a arquitetura de *Backend as a Service (BaaS)* e o roadmap de infraestrutura discutidos para o projeto **Sudoku Aneróthico PRÓ**, garantindo escalabilidade gratuita, conformidade legal e segurança de dados.

1. Arquitetura de Dados (Zero Custo Serverless)

A aplicação adotará um modelo descentralizado focado na otimização de requisições e custos da camada gratuita.

- **Cache L1 / Hot Storage (Firebase Realtime Database):**
  - Focado em velocidade e dados voláteis (partidas ativas, chat em tempo real, status online).
  - **Data Flattening:** Separação estrita entre informações vitais do usuário (ex: vitórias, rank, apelido) num nó principal e o histórico de partidas noutro nó para viabilizar *lazy loading* sem baixar toda a árvore de dados do usuário.
  - **Otimização de Payload:** Partidas gravadas apenas com a *seed* de geração, dificuldade e tempo, eliminando arrays completos de 81 células.
- **Cache L2 / Cold Storage (Google Drive via Apps Script):**
  - Armazenamento de longo prazo (backups, logs de auditoria e usuários inativos) em formato JSON.
  - Gerenciado por uma conta Google exclusiva de desenvolvedor (15 GB limpos).
- **Restauração Sob Demanda (Web App Endpoint):**
  - Se um usuário inativo (dados expurgados do Firebase para o Drive) fizer login, o cliente fará um POST para um endpoint REST (criado no Google Apps Script). O script buscará o arquivo JSON no Drive e regravará no Firebase instantaneamente, eliminando a espera por ciclos diários do cron job.

2. Automação e Jobs Assíncronos (Cold/Lazy Jobs)

A manutenção e limpeza do banco de dados ocorrerão sem a necessidade de um servidor backend 24/7.

- **Google Apps Script (Cron Jobs em Nuvem):**
  - Responsável pela execução diária (ex: durante a madrugada) das rotinas de expurgo de salas inativas.
  - Faz o *dump* de dados do Firebase (REST API), converte para JSON e salva nativamente no Google Drive (Cold Storage).
  - Deleta dados defasados do Google Drive após expirar o prazo de retenção (ex: logs de 180 dias).
- **Aplicação Master (Painel de Administração Local):**
  - Terminal/interface web de uso restrito do desenvolvedor, hospedada e rodando localmente (similar ao atual script do *make interativo*).
  - Lê variáveis de controle (ex: `backup_interval_days`) e dispara *jobs* ou altera os intervalos do Apps Script via alteração de estado num nó oculto no Firebase (`/admin_config`).

3. Conformidade Legal e Chat Seguro

A estratégia de comunicação foi desenhada para blindar a aplicação de abusos e respeitar a Lei Geral de Proteção de Dados (LGPD) e o Marco Civil da Internet (MCI).

- **Coleta e Retenção para o Marco Civil (Art. 15):**
  - Coleta do endereço IP, *timestamp* UTC e fuso horário no momento do login.
  - Armazenamento em cold storage (Google Drive) pelo prazo estrito de **180 dias** (6 meses). Após esse período, o script de limpeza exclui os arquivos do Drive.
- **Aviso de Privacidade e Consentimento:**
  - O usuário deverá concordar com os Termos no momento da criação da conta, tomando ciência explícita de que seu IP está sendo guardado por razões de obrigatoriedade legal.
- **Sistema Híbrido de Comunicação (Mitigação de Abuso):**
  - **Usuários Anônimos:** Apenas "Quick Chat" (mensagens parametrizadas/pré-fabricadas como "Acho que a célula R2C3 está errada"). O tráfego será feito via identificadores minúsculos (`{ msgId: 12 }`), que reduzem drasticamente o payload no Firebase e eliminam

riscos de assédio ou toxicidade, pois o usuário não pode escrever texto livre.
    - **Usuários Autenticados:** Acesso ao chat de texto livre.

4. Sistema de Autenticação Customizada

Para evitar os custos ocultos e a complexidade de logins federados (ex: Google Auth), a autenticação será tratada nativamente usando o Firebase.

- **Conta Obrigatória para Features Premium:** O acesso ao histórico cruzado entre dispositivos e ao chat de texto livre exigirá login.
- **Credenciais Requeridas:**
    - E-mail válido (sem exigência imediata de confirmação/double opt-in para jogar, mas a sintaxe e o formato serão validados).
    - Senha.
- **Segurança (Hashes):** Utilização da Web Crypto API do navegador para gerar *hash* da senha antes de ser transmitida, comparando a string encriptada nas Firebase Security Rules para autorizar a escrita/leitura no nó do usuário.

5. Práticas de Engenharia e DevOps (Portfólio)

- **CI/CD Pipeline (GitHub Actions):**
    - Configuração de esteira para executar testes unitários e de integração a cada `push`.
    - **Branch Protection Rules:** Bloqueio de commits diretos na branch `main`. Apenas *Pull Requests* com testes verdes na automação (*Passing*) poderão ser aprovados, garantindo a robustez do repositório público para análise de recrutadores.
- **Segurança de Credenciais:** As chaves de serviço de APIs e configurações do Firebase nunca serão commitadas no código público, residindo nos Secrets do GitHub e em variáveis de ambiente `.env` locais.